

Project ORCA

Vision-Based Biomimetic Control of a 16-DOF Robotic Hand



Technical Report

Author:
LEROY Gabin

Supervisor:
TOGO Shunta

Academic Year 2025-2026

Contents

1	Hardware Architecture	3
1.1	Vision Recognition System: Raspberry Pi & OAK-D	3
1.1.1	OAK-D Lite Spatial Camera	3
1.1.2	Processing Unit: Raspberry Pi 5	3
1.2	Actuation System: The ORCA Hand	4
1.2.1	Actuation Mechanism: Antagonistic Tendon System	4
1.2.2	Low-Level Controller: Teensy Microcontroller	6
1.2.3	Power Distribution: PCA9685 and Battery Management	6
2	Software Implementation	7
2.1	Visual Acquisition with DepthAI	7
2.2	Hand Tracking: OpenCV and MediaPipe	8
2.3	Kinematics and Signal Processing	10
2.3.1	Joint Angle Calculations	10
2.3.2	Jitter Reduction: Deadband Filter Implementation	13
2.3.3	Data Normalization: PWM Mapping and Inversions	13
2.4	Communication Protocol and Firmware	15
2.4.1	Serial Framing Protocol	15
2.4.2	Teensy Serial Parser	15
3	System Integration and Challenges	16
3.1	Architecture Optimization: Direct I2C Communication	16
3.2	Hybrid Mode: Manual MIDI Override	16
3.2.1	Control Arbitration and Mode Toggling	16
3.2.2	Physical Mapping Interface	16
3.3	Vision Integration Challenge: DepthAI Ecosystem	17
3.4	Biomechanical Safety Constraints	17
3.5	Electrical and Buffer Optimization	18
3.6	System Validation and Experimental Results	18
4	Conclusion and Future Work	23
4.1	Possibilities for Improvement	23
4.1.1	Advanced Tracking and Multi-Camera Fusion	23
4.1.2	Algorithmic Optimization and Inverse Kinematics	23
4.1.3	Hardware Enhancements and Tactile Feedback	23
4.2	Final Summary	24
5	Appendices	25
5.1	Kinematics and Geometric Core Functions	25
5.2	Abduction and Sign Determination Algorithm	26
5.3	Thumb Opposition Algorithm	27
5.4	Multi-Joint Deadband Filter Algorithm	28
5.5	Python Data Sorting and PWM Mapping Algorithms	28
5.6	Teensy Calibration and Hardware Interfacing Firmware	30
5.7	Python Serial Transmitter Pipeline (Raspberry Pi)	31

5.8	Teensy Serial Receiver and Parser Firmware	32
5.9	Teensy MIDI Manual Override Firmware	33
6	Bibliography	35

1. Hardware Architecture

1.1 Vision Recognition System: Raspberry Pi & OAK-D

1.1.1 OAK-D Lite Spatial Camera

The visual sensing unit of the ORCA project is the OAK-D Lite (OpenCV AI Kit with Depth), as shown in Figure 1.1. This device is a **spatial AI camera** powered by the **Intel Myriad X VPU**, which allows it to perceive its environment in three dimensions while processing complex neural networks locally.

Unlike standard monocular cameras, the OAK-D Lite features a **trinocular sensor configuration**:

- **Dual Mono Sensors:** Two synchronized global shutter cameras that provide the **stereoscopic vision** necessary to generate a real-time disparity map.
- **4K RGB Camera:** A central high-resolution sensor dedicated to visual recognition and color stream acquisition.



Figure 1.1: The OAK-D Lite Spatial AI Camera

In the context of the ORCA hand, this spatial capability enables the system to distinguish the user's hand from the background with high precision and provides accurate **3D coordinates** (X, Y, Z) for each detected landmark. The combination of the RGB stream for AI inference and the mono sensors for depth tracking ensures a robust control signal.

1.1.2 Processing Unit: Raspberry Pi 5

The central coordination of the ORCA system is handled by a **Raspberry Pi 5**. This board provides the high processing power and connectivity required for the project. Although an **OLED screen** is physically mounted on the Pi board (as shown in Figure 1.2), it was not utilized in the final software implementation. Since rendering a real-time stream on an auxiliary display was a secondary priority, development time was focused entirely on optimizing the core tracking pipeline and reducing system latency. Instead, visual monitoring is handled directly on the main operating system display via OpenCV.

In this architecture, the Raspberry Pi acts as the bridge between the vision input and the mechanical output. It is physically connected to the OAK-D Lite camera (via

USB) and to the Teensy microcontroller (via Serial). Its software pipeline performs the following sequence:

1. **Landmark Recognition:** It processes the hand landmarks captured by the camera.
2. **Visual Monitoring:** It displays the live tracking feedback on the main desktop monitor using OpenCV.
3. **Data Calculation:** It converts the spatial position of the landmarks into joint angles, and then maps these angles into PWM (Pulse Width Modulation) values.
4. **Transmission:** It sends these final values to the Teensy for motor execution.



Figure 1.2: Raspberry Pi 5 with the OLED screen

1.2 Actuation System: The ORCA Hand

1.2.1 Actuation Mechanism: Antagonistic Tendon System

The mechanical movement of the ORCA hand is powered by **16 MG90D digital servomotors**. These motors were primarily selected for their **affordability**, while offering a **270-degree rotation range**, which is more versatile than standard 180-degree servos for joint actuation.

Each joint is moved via a **tendon-driven system** (high-strength strings) connected to the servos, as shown in Figure 1.3.

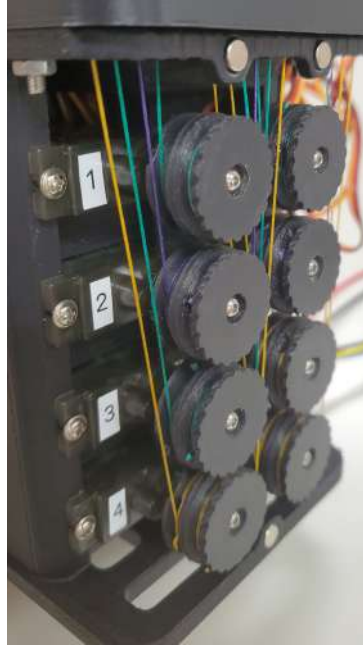


Figure 1.3: Internal assembly with the MG90D servos and tendon routing

The mechanism uses a **dual-pulley system** to convert rotation into linear tension. This "push-pull" configuration (Figure 1.4) ensures constant control in both directions:

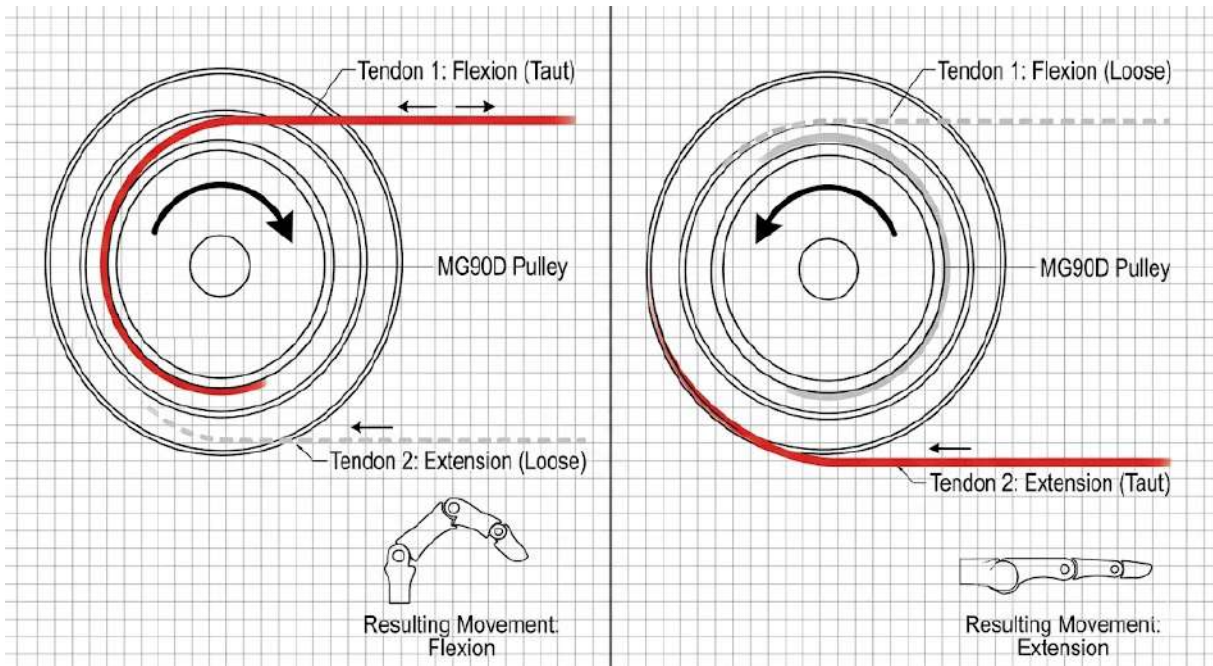


Figure 1.4: Antagonistic Pulling System with Flexion (left) and Extension (right) operational logic

As the servo rotates, one tendon becomes **taut** (Active Pull) while the other loosens. This antagonistic setup allows for precise joint positioning without string slack, maintaining mechanical accuracy throughout the movement range.

1.2.2 Low-Level Controller: Teensy Microcontroller

The Teensy is used to send the PWM signals to the servomotors. It receives the commands from the Raspberry Pi via **Serial** communication and then transmits them to the PCA9685 driver using the **I2C** protocol.

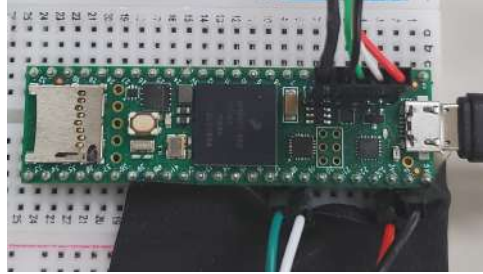


Figure 1.5: Teensy microcontroller board acting as a bridge for motor control

1.2.3 Power Distribution: PCA9685 and Battery Management

The PCA9685 acts as the central hardware distributor for both power and signals for the 16 servomotors. Its role is to convert digital commands received via the **I2C** bus into physical **PWM** signals sent individually to each motor.

To support the electrical load of the 16 actuators, the module features a dedicated power terminal block connected directly to the battery. This architecture enables electrical isolation between the logic power supply (Teensy) and the actuator power supply (motors). This separation is essential to protect the control electronics from electromagnetic noise and voltage drops caused by the mechanical activity of the servomotors.

2. Software Implementation

To ensure stability and reproducibility on the Raspberry Pi 5, the following environment setup must be performed before running the control scripts:

- **Python Version:** The project is built using **Python 3.11.9** for full library compatibility.
- **Virtual Environment:** A dedicated environment is used to manage dependencies. It is created with `python -m venv VenvName` and activated via:
`source VenvName/bin/activate`.
- **Library Dependencies:** The following versions are required for the system to operate correctly:
 - **numpy 1.26.4:** Used for array manipulation and mathematical operations.
 - **depthai 2.24.0:** Framework required to interface with and initialize the OAK-D camera.
 - **opencv-python 4.11.0.86:** Used for image processing and visual feedback.
 - **mediapipe 0.10.18:** Framework for real-time hand landmark tracking.
 - **pyserial 3.5:** Protocol used to send the calculated data to the Teensy micro-controller.

2.1 Visual Acquisition with DepthAI

The OAK-D Lite is not a standard webcam; it functions as an independent processing unit. To capture images, we must define a **Pipeline**, a logical map of operations executed directly on the camera's VPU. This allows the hardware to handle resizing and color conversions internally, maintaining a high frame rate on the Raspberry Pi.

As illustrated in Listing 2.1, the pipeline configuration optimizes the data stream directly on the hardware layer.

```
1 # 1. Initialize the pipeline object
2 p = dai.Pipeline()
3
4 # 2. Define the building blocks (Nodes)
5 cam = p.create(dai.node.ColorCamera) # Access the physical RGB
   sensor
6 xout = p.create(dai.node.XLinkOut)    # Create the USB output
   channel
7 xout.setStreamName("video")           # Name the stream for the
   Raspberry Pi
8
9 # 3. Hardware-level sensor configuration
10 cam.setBoardSocket(dai.CameraBoardSocket.RGB) # Use the central RGB
   camera
11 cam.setResolution(dai.ColorCameraProperties.SensorResolution.
   THE_1080_P) # Set 1080p
```



```

12 cam.setPreviewSize(640, 480) # Resize now to optimize MediaPipe
    speed for later
13 cam.setInterleaved(False)    # Use planar format for VPU efficiency
14 cam.setColorOrder(dai.ColorCameraProperties.ColorOrder.BGR) # Set
    BGR order for later OpenCV compatibility
15
16 # 4. Finalize the data path
17 cam.preview.link(xout.input) # Link the resized preview to the USB
    output

```

Listing 2.1: DepthAI Pipeline and Node Configuration

By setting the `previewSize` and `colorOrder` directly on the hardware, the Raspberry Pi receives a stream that is already optimized for the subsequent software layers. This is a critical design choice: it ensures that the image is "ready to use" for **MediaPipe** and **OpenCV**, avoiding the need for expensive software-based resizing or color conversions that would otherwise saturate the Pi's CPU.

2.2 Hand Tracking: OpenCV and MediaPipe

Since the camera was configured in the previous section, the frames can now be displayed on the screen using OpenCV (Figure 2.1). This live feed allows continuous monitoring of the system status and ensures the user's hand remains within the camera's field of view.

The following implementation (Listing 2.2) details how the frames are retrieved and rendered:

```

1 # 1. Retrieve the frame from the DepthAI output queue (blocking
    call)
2 in_video = q.get()
3 img = in_video.getCvFrame() # Reconstruct the packet into a NumPy
    pixel matrix
4
5 # 2. Display the image in a dedicated window
6 cv2.imshow("Projet ORCA - Vision", img)
7
8 # 3. Refresh the UI and listen for keyboard input
9 key = cv2.waitKey(1) # Wait 1ms for a key press to keep the window
    responsive
10 if key == ord('q'): # Emergency exit command
11     break
12
13 # 4. Cleanup resources after the loop ends
14 cv2.destroyAllWindows()

```

Listing 2.2: Camera feedback and UI management with OpenCV

The `getCvFrame()` command serves as a bridge that reconstructs the raw data packets into a structured **NumPy array**. In this format, the image is treated as a multi-dimensional matrix where every entry represents a specific pixel's color values. This structure is essential for the subsequent processing stages, as it allows mathematical frameworks to perform operations directly on the image data.

Furthermore, the use of `cv2.waitKey(1)` is mandatory for the display loop; it provides the necessary time for the operating system to render the image on the screen while simultaneously listening for user commands, such as the emergency exit or debug triggers. This stable visual base is what allows the **MediaPipe** framework to then scan the image for hand landmarks.



Figure 2.1: Camera's feedback via OpenCV

Once the visual stream is stabilized, the project utilizes the **MediaPipe** framework to perform real-time hand tracking. While OpenCV handles the display, MediaPipe acts as the analytical layer that extracts a digital skeleton from the pixel matrix.

To ensure accurate detection, the system follows the processing sequence detailed in Listing 2.3. The first step is a color space conversion, as MediaPipe's neural networks are optimized for **RGB** input, whereas OpenCV provides frames in **BGR** format.

```

1 # 1. Initialize the Hands model with a 70% detection confidence
2 hands = mp_hands.Hands(max_num_hands=1, min_detection_confidence
3     =0.7)
4
5 # 2. Convert the frame to RGB for compatibility
6 img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
7
8 # 3. Process the image to detect landmarks
9 res = hands.process(img_rgb)
10
11 # 4. Draw the skeleton if a hand is detected
12 if res.multi_hand_landmarks:
13     for lm in res.multi_hand_landmarks:
14         # Draw both the 21 points and the connecting lines
15         mp_draw.draw_landmarks(img, lm, mp_hands.HAND_CONNECTIONS)

```

Listing 2.3: Hand Landmark detection and skeleton extraction

As illustrated in the theoretical model (Figure 2.2), the framework identifies **21 distinct 3D landmarks**. To form a coherent digital skeleton, the system draws lines be-

tween specific points using a predefined connection map (`HAND_CONNECTIONS`), resulting in the real-time tracking shown in Figure 2.3.

Each point is defined by (x, y, z) coordinates that are **normalized** relative to the image dimensions. This ensures that the tracking remains consistent regardless of the camera resolution or the user's distance. These normalized coordinates provide the numerical foundation required for the subsequent joint angle calculations.

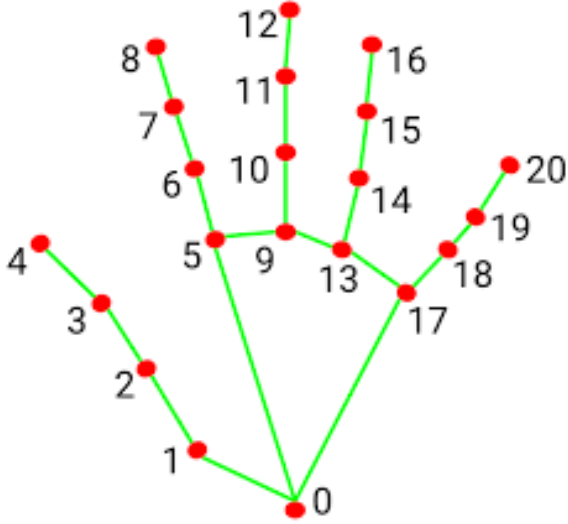


Figure 2.2: MediaPipe Hand Landmark model (21 points)

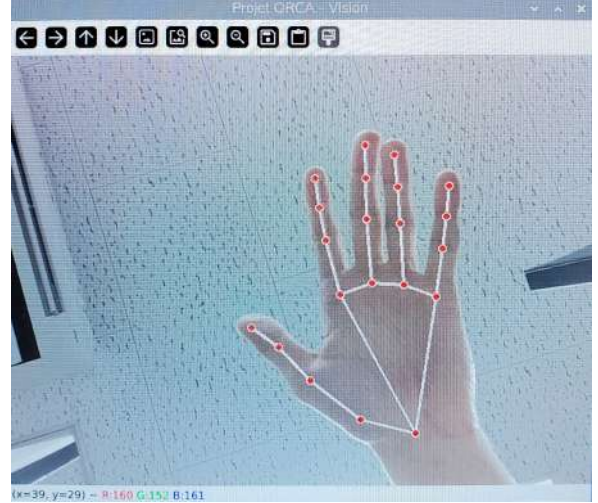


Figure 2.3: Real-time landmark extraction result

2.3 Kinematics and Signal Processing

2.3.1 Joint Angle Calculations

Now that the spatial coordinates of the hand skeleton are captured, the system must calculate the specific angle for each mechanical joint. The ORCA hand features four types of joint movements; therefore, the first step is to establish a generic geometric method to compute them.

Instead of using complex inverse kinematics, the system leverages direct **geometric vector analysis**. Since MediaPipe provides the exact 3D coordinates for every point, the angle θ between any two directional vectors $\vec{u}$ and $\vec{v}$ can be determined by isolating the angle from the dot product formula, as defined in Equation 2.1:

$$\theta = \arccos \left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \times \|\vec{v}\|} \right) \quad (2.1)$$

The simplest movements to track are the finger flexions. Mechanically, the ORCA hand features a simplified structure where the distal phalanx is fixed and cannot move independently. To achieve a natural grasping motion despite this structural constraint, an intentional index offset was implemented in the software mapping logic.

Instead of a direct joint-to-joint replication, the calculated human distal flexion angle (derived from landmarks 6, 7, and 8 as shown in Figure 2.5) is mapped to drive the

robotic middle joint. Concurrently, the human middle flexion angle (derived from landmarks 5, 6, and 7 as shown in Figure 2.4) is used to command the robotic proximal joint. Empirical testing showed that this adaptation significantly improves the enfolding behavior of the mechanical fingers. The complete algorithmic implementation of these vector transformations is detailed in Appendix 5.1.

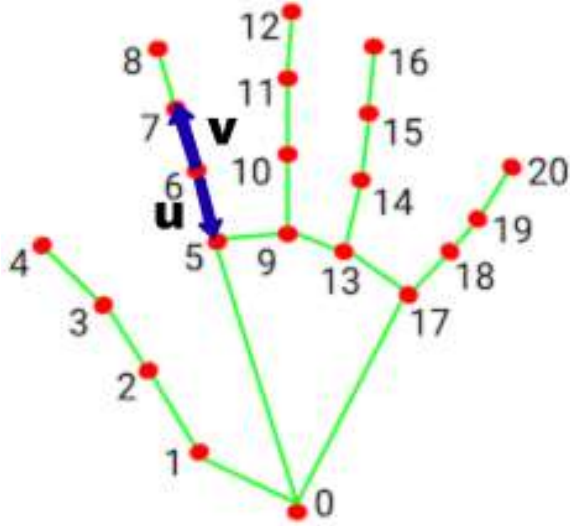


Figure 2.4: Vectors used for the proximal joint flexion (landmarks 5, 6, 7)

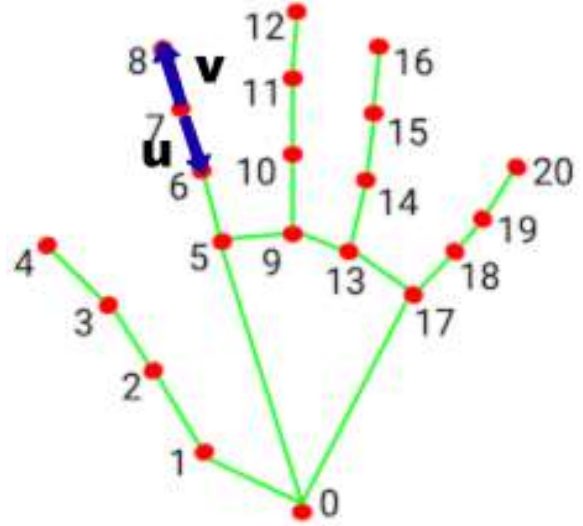


Figure 2.5: Vectors used for the distal joint flexion (landmarks 6, 7, 8)

Unlike flexion, tracking finger **abduction** requires a global reference axis rather than consecutive segments. As shown for the index finger in Figure 2.6, the algorithm computes the angle between the palm reference vector $\vec{u}_{ref}$ (from wrist landmark 0 to middle finger base landmark 9) and the finger directional vector $\vec{v}_{finger}$ (landmarks 5 to 6 for the index). This angle is isolated using the same dot product framework defined in Equation 2.1.

A unique challenge arises with the middle finger, which defines the hand's centerline and can abduct symmetrically to both sides. Since the standard arccos function only returns unsigned values (0° to 180°), a **2D determinant** (planar cross product) is calculated using the X and Y coordinates of the two vectors. The sign of this determinant dictates the direction of the lateral deviation, allowing the system to output a signed integer. The complete implementation is provided in Appendix 5.2.

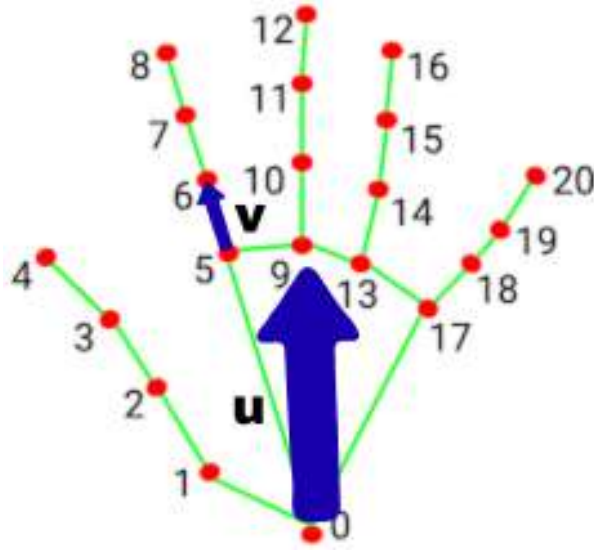


Figure 2.6: Vectors used for joint abduction (landmarks 0, 9 and 5, 6)

Finally, tracking the unique biomechanical movement of the thumb requires a dedicated metric called **opposition**, which measures the thumb's ability to sweep across the palm towards the little finger. As illustrated in Figure 2.7, the algorithm constructs a reference baseline vector $\vec{u}$ between the base of the thumb (landmark 1) and the base of the little finger (landmark 17). This baseline is then compared against the thumb's directional vector $\vec{v}$, mapped from its base to its first interphalangeal joint (landmarks 1 to 2). The opposition angle is isolated using the same dot product framework defined in Equation 2.1.

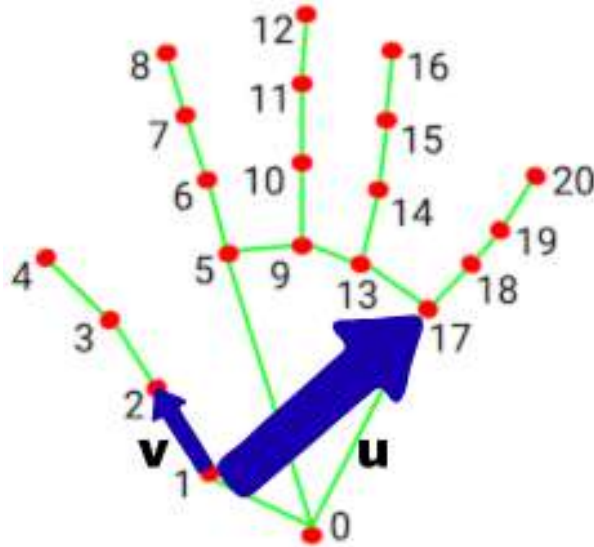


Figure 2.7: Vectors used for thumb opposition (landmarks 1, 17 and 1, 2)

Once all joint angles (flexions, abductions, and opposition) are calculated by the vision pipeline, they are compiled into a structured array sequence. The sorting algorithm and PWM mapping logic are detailed in Section 2.3.3. The complete algorithmic implementation for the thumb opposition calculation is provided in Appendix 5.3.

2.3.2 Jitter Reduction: Deadband Filter Implementation

Raw spatial data extracted from computer vision frameworks inherently suffers from microscopic fluctuations caused by pixel noise, lighting variations, and floating-point roundings. If these raw coordinates were transmitted directly to the ORCA robotic hand, the servomotors would jitter continuously, leading to high power consumption, overheating, and mechanical wear. To mitigate this issue, a software-level stabilization filter is introduced.

The system implements a **deadband filter** (or threshold zone) that acts as a mathematical buffer between consecutive frames. For each individual joint angle, the algorithm compares the newly calculated angle against the previously validated angle. If the absolute difference between these two values is less than or equal to a strict threshold of 3° , the change is treated as background noise, and the sent angle remains unchanged. The servomotor position is updated only when a physical movement exceeds this 3° deadband zone. This simple yet highly effective process ensures smooth mechanical behavior without introducing noticeable latency. The programmatic implementation of this multi-joint filter is provided in Appendix 5.4.

2.3.3 Data Normalization: PWM Mapping and Inversions

Once the joint angles are stabilized by the deadband filter, they cannot be transmitted blindly to the hardware. Two final software adaptations are mandatory: reordering the data structure to match the physical wire layout of the hand, and scaling geometric angles into specific Pulse Width Modulation (PWM) signals.

The first step is executed by the sorting algorithm (`sort_ORCA`). Because the mathematical computation sequence does not natively match the physical channel order (0 to 15) on the PCA9685 driver board, the calculated angles must be systematically reindexed. This alignment ensures that the data package received by the low-level controller addresses the correct physical fingers.

Subsequently, each sorted angle must be translated into a hardware-compatible PWM value. To perform this, extensive empirical calibration was conducted using a custom firmware script adapted from the supervisor’s initial framework (detailed in Appendix 5.6). This tool allowed for direct interaction with the 12-bit PCA9685 driver via I2C to determine the exact structural limits of the 16 digital MG90D servomotors.

For each individual joint, two critical boundary pairs were mapped:

- **Biomechanical Range:** The minimum and maximum calculated angles (in degrees) that the vision system captures during natural hand movements.
- **Actuation Range:** The safe minimum and maximum PWM registers preventing mechanical binding, gear strain, or tendon slack.

As illustrated in Figure 2.8 and Figure 2.9, every single joint possesses distinct boundary values due to manual assembly tolerances and specific tendon-routing pathways.

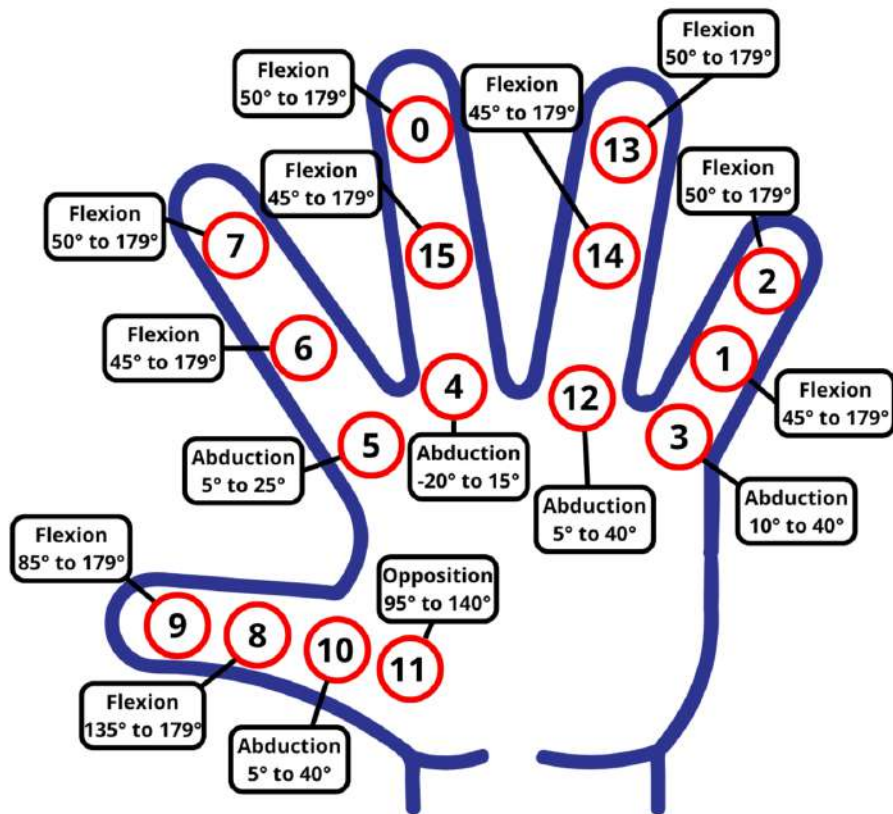


Figure 2.8: Empirical biomechanical angle boundaries (degrees) per joint

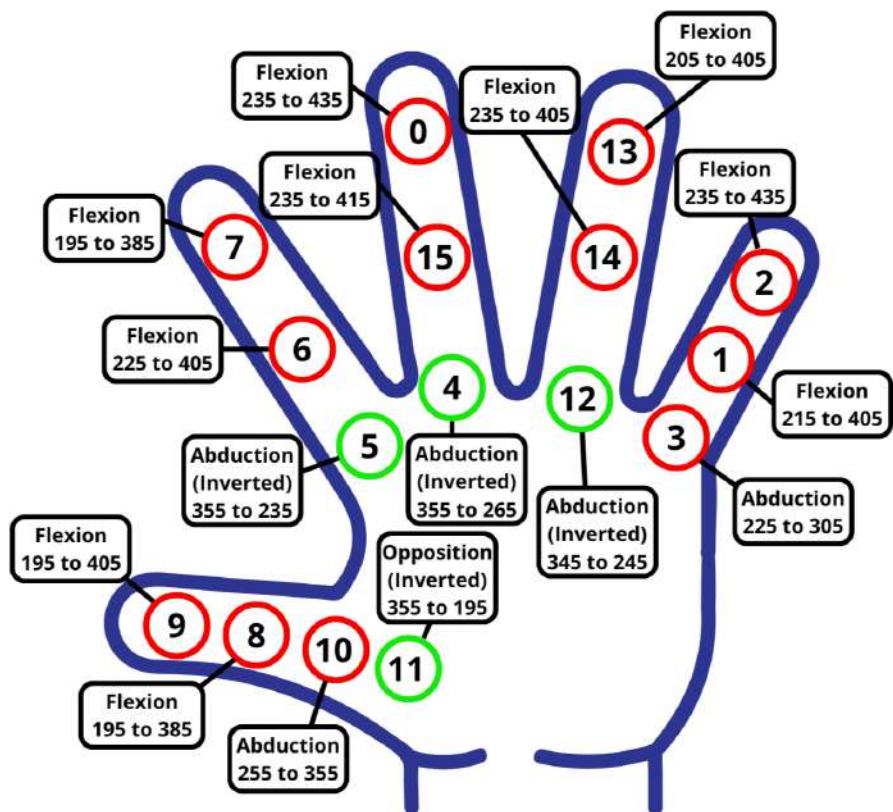


Figure 2.9: Calibrated physical PWM control boundaries per joint

Using these dual-boundary maps, the software leverages the `np.interp` function to perform linear regressions. It is worth noting that for certain joints, the mapping boundaries are inverted (e.g., mapping $[95^\circ, 140^\circ]$ to $[355, 195]$). This inversion is a design requirement to compensate for the mirrored physical placement of the servomotors inside the ORCA hand frame, ensuring that a mathematical extension always results in a physical extension. The complete normalization and calibration algorithms are provided in Appendix 5.5. To verify these processing steps during development, a diagnostic keyboard shortcut was included in the Python loop. Pressing the 't' key prints the raw calculated angles, the filtered angles, and the final PWM values directly into the terminal for real-time monitoring.

2.4 Communication Protocol and Firmware

This section details the data transmission between the Python pipeline and the microcontroller, focusing on the framing protocol and the serial parsing architecture.

2.4.1 Serial Framing Protocol

To transmit the calculated data over the USB serial link, a packet-framing protocol prevents data corruption and loss of synchronization. Each data frame is encapsulated between a start marker '<' and an end marker '>'. The payload consists of the 16 PWM integers, ordered to match the PCA9685 driver channels, and separated by commas:

$$\langle \text{PWM}_0, \text{PWM}_1, \text{PWM}_2, \dots, \text{PWM}_{15} \rangle \quad (2.2)$$

The microcontroller discards any incoming bytes until a valid start marker is detected, ensuring that incomplete or corrupted packets do not trigger motor movements. The Python transmission code running on the Raspberry Pi is detailed in Appendix 5.7.

2.4.2 Teensy Serial Parser

To avoid loop latency, the Teensy firmware processes incoming serial data byte-by-byte within the main execution loop, avoiding synchronous blocking functions.

The parsing sequence operates as follows:

- **Frame Synchronization:** The parser ignores all characters until '<' resets the numeric buffer and the channel index counter.
- **Buffer Accumulation:** Numerical characters are sequentially appended to an internal string buffer.
- **Value Extraction:** When a comma or the '>' marker is encountered, the accumulated string is converted into a standard integer using the native `toInt()` method and assigned to its corresponding servo index.

The complete microcontroller serial parsing implementation is provided in Appendix 5.8.

3. System Integration and Challenges

3.1 Architecture Optimization: Direct I2C Communication

To simplify the system architecture, the Teensy microcontroller was removed, establishing a direct I2C connection between the Raspberry Pi 5 and the PCA9685 driver board. This optimization reduces the hardware footprint and runs the entire pipeline within a single Python 3.11.9 environment using the `adafruit-circuitpython-pca9685` library. The physical wiring connects the PCA9685 directly to the primary I2C pins of the Raspberry Pi 5: Pin 1 (3.3V), Pin 3 (SDA), Pin 5 (SCL), and Pin 7 (Ground), operating at the hardware address `0x40` at 50 Hz.

This direct layout required adapting the data resolution and migrating the low-level safety features into the main Python execution loop:

- **12-to-16 Bit Resolution Scaling:** The tracking framework outputs 12-bit PWM values (0 to 4095), but the CircuitPython library requires a 16-bit integer for the `.duty_cycle` attribute. Each value in the `PWM_to_send` array is programmatically multiplied by 16 and cast to an integer to match the hardware requirements without precision loss.
- **Migration of Hardware Safety Guardrails:** The safety routines previously handled by the Teensy firmware were adapted directly into the Python script. To prevent battery current spikes, a 2 ms sequential delay (`time.sleep(0.002)`) is introduced between individual servo updates inside the loop. Additionally, a vision-based power cutoff was added: if no hand is detected by MediaPipe, the script instantly forces the `duty_cycle` of all 16 channels to 0, completely relaxing the tendons and cutting idle motor power.

3.2 Hybrid Mode: Manual MIDI Override

To facilitate hardware debugging and empirical calibration, a secondary control layer based on the MIDI (Musical Instrument Digital Interface) protocol was integrated, allowing the operator to bypass the vision pipeline.

3.2.1 Control Arbitration and Mode Toggling

The firmware boots in AI Mode by default. When the operator presses the selection button (CC 41) highlighted in Figure 3.1, the Teensy toggles the `isManualMode` flag. In manual mode, the microcontroller flushes the incoming serial hardware buffer continuously to prevent data accumulation from freezing the Raspberry Pi pipeline.

3.2.2 Physical Mapping Interface

The KORG nanoKONTROL2 layout is configured to mirror the structural breakdown of the ORCA hand:

- **Sliders 1 to 5:** Control the proximal phalanges (flexion 1) from the thumb to the little finger.
- **Knobs 1 to 5:** Control the distal and middle phalanges (flexion 2) from the thumb to the little finger.
- **Remaining Controls:** Map the independent abduction and opposition joints.

The exact correlation between these physical control lines, their internal channels, and the physical joints of the robotic fingers can be verified by cross-referencing the physical index map detailed in Figure 2.9.



Figure 3.1: Physical control mapping layout of the Korg nanoKONTROL2 surface.

The dedicated stop/reset button (CC 42), visible on the control map (Figure 3.1), triggers an instant recalibration routine, resetting all joints to their neutral positions. Additionally, a firmware safety timeout automatically writes a value of 4096 to the PCA9685 registers if a servo remains stationary for more than 500 ms, cutting off power delivery to prevent motor overheating. The manual control override and MIDI callback logic are provided in Appendix 5.9.

3.3 Vision Integration Challenge: DepthAI Ecosystem

A practical challenge during the initial phase was interfacing with the OAK-D Lite camera. Unlike a standard USB webcam initialized with basic open-source functions, this spatial camera functions as an independent edge-processing unit.

Operating the device required learning and implementing the proprietary `depthai` library. This architecture requires building a dedicated hardware pipeline, defining separate camera and output nodes that execute directly on the camera’s onboard processor. Navigating this ecosystem was necessary to achieve hardware-level image resizing and configuration before transmitting frames to the MediaPipe tracking script.

3.4 Biomechanical Safety Constraints

Operating a high-torque 16-DOF robotic hand presents structural risks; excessive servo rotations can instantly snap the internal tendon lines or fracture the 3D-printed finger links. To guarantee mechanical safety, two redundant layers of soft-boundaries were hard-coded into the software control loop:

- **High-Level Constraints:** The Python tracking script applies the `np.clip()` function on raw coordinates, bounding the software’s geometric angles within safe biological limits before execution.
- **Low-Level Guardrails:** The Teensy firmware implements strict internal conditional clamps inside its command routine. If a corrupted or out-of-bounds register value passes through the serial link, the micro-controller overrides the data, clipping the output pulse width strictly between a minimum of 50 and a maximum of 600.

3.5 Electrical and Buffer Optimization

Actuating the 16 digital MG90D servomotors simultaneously creates severe electrical constraints. Using a battery pack equipped with an autonomy display, empirical testing revealed that power consumption drops drastically with each active motor. Specifically, when approximately 5 or more servos are active under mechanical load at the same time, the current draw triggers the battery’s internal safety circuit, completely cutting off the power supply. To reduce this electrical strain and prevent power shutdowns, two optimization strategies were implemented:

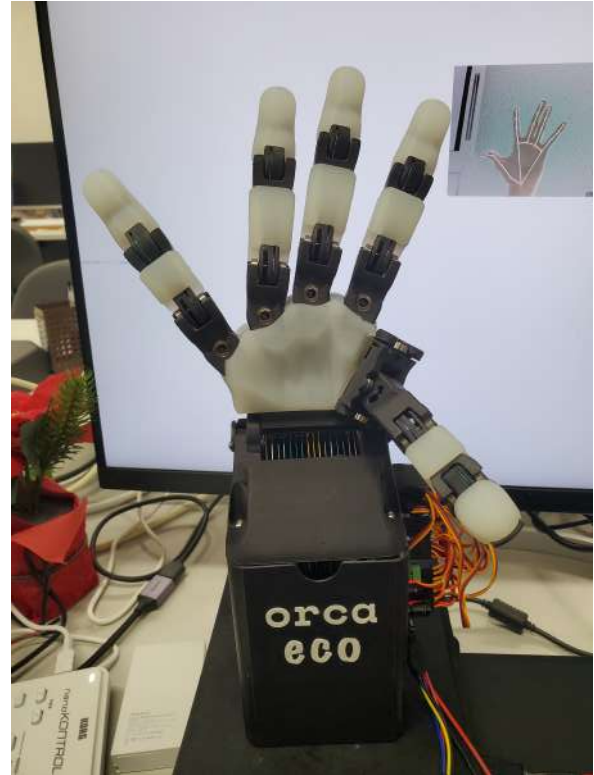
1. **Cascade Execution Delay:** When rewriting the 16 channel states, the Teensy introduces a minor sequential loop delay of 2 milliseconds between individual driver updates. This micro-staggering layout spreads the initial servo startup currents over time, effectively flatlining hazardous amperage spikes from the battery pack.
2. **Asynchronous Power Cutoff:** When a robotic finger remains stationary for more than 500 milliseconds, the firmware automatically writes a state value of 4096 to that specific PCA9685 register. This code entirely disengages the PWM signal, cutting off current delivery to eliminate motor buzzing, idle power consumption, and thermal overheating.

3.6 System Validation and Experimental Results

To validate the system, experimental tests were conducted using several different hand postures. For each test, the visual tracking input captured by the camera is compared directly with the physical posture reproduced by the robotic hand, confirming that the teleoperation pipeline functions correctly.



(a) MediaPipe tracking input



(b) Physical robotic replication

Figure 3.2: Fully open hand posture configuration



(a) MediaPipe tracking input

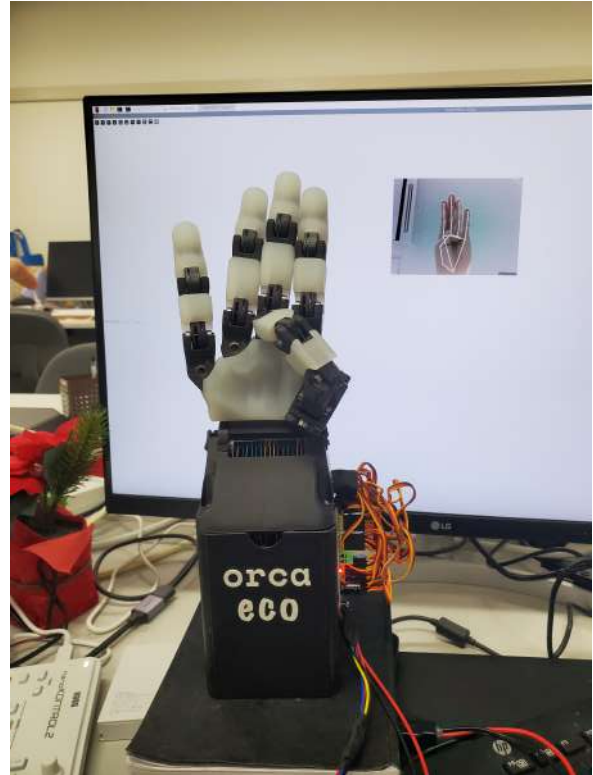


(b) Physical robotic replication

Figure 3.3: Flat hand posture with adducted fingers



(a) MediaPipe tracking input



(b) Physical robotic replication

Figure 3.4: Adducted fingers with the thumb tucked inside the palm



(a) MediaPipe tracking input



(b) Physical robotic replication

Figure 3.5: Closed fist posture with an external thumb lock

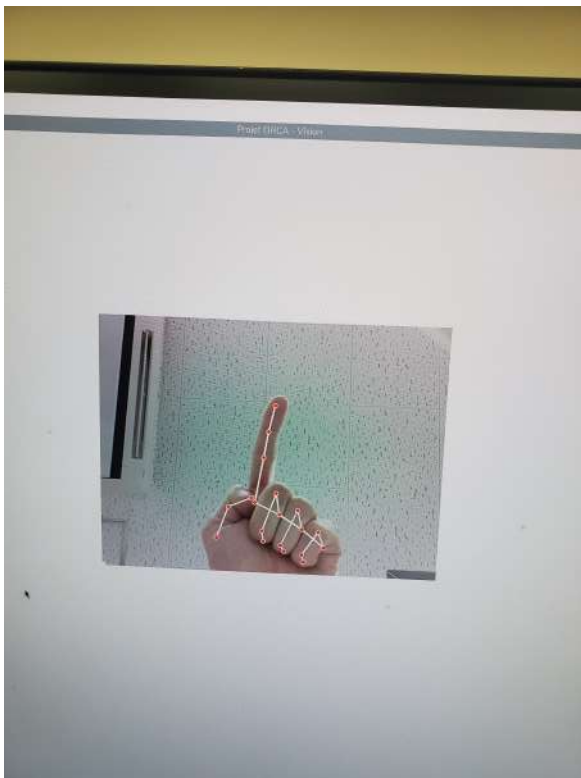


(a) MediaPipe tracking input

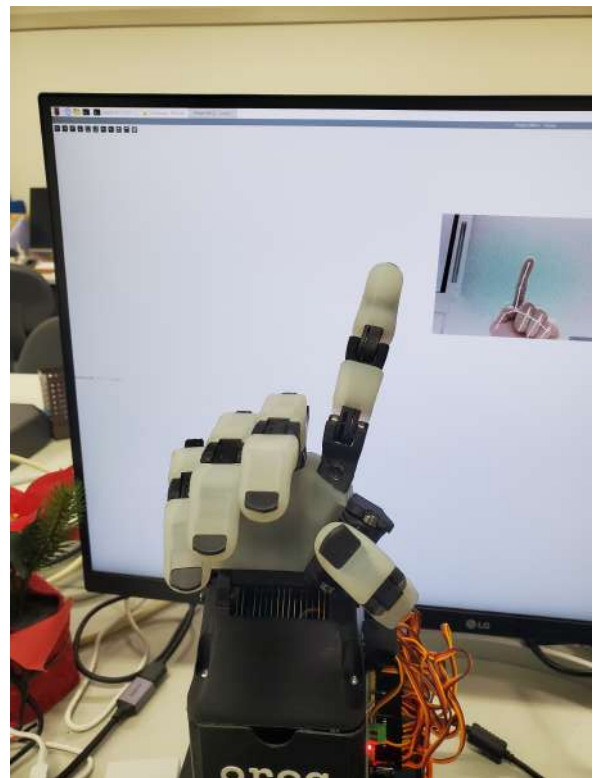


(b) Physical robotic replication

Figure 3.6: Closed fist posture with an internal tucked thumb



(a) MediaPipe tracking input



(b) Physical robotic replication

Figure 3.7: Closed fist posture with an extended index finger (pointing gesture)



(a) MediaPipe tracking input



(b) Physical robotic replication

Figure 3.8: Closed fist posture with extended index and middle fingers (victory gesture)

4. Conclusion and Future Work

4.1 Possibilities for Improvement

Based on the empirical limitations observed during testing, several realistic software and hardware upgrades are proposed to improve the tracking precision and functional capabilities of the ORCA hand.

4.1.1 Advanced Tracking and Multi-Camera Fusion

The current tracking system suffers from coordinate drifting during hand occlusions, particularly when the hand is fully closed. Two alternative approaches can mitigate this issue:

- **Targeted AI Model Training:** Training a specialized neural network on a dedicated dataset of closed and occluded hand gestures represents the most viable solution. It optimizes tracking precision while keeping system complexity and financial costs low.
- **Multi-Camera Array:** Implementing a multi-view camera configuration would resolve perspective occlusion by fusing 3D coordinate inputs from multiple angles. However, this method significantly increases hardware costs and processing requirements.

4.1.2 Algorithmic Optimization and Inverse Kinematics

The current implementation relies on separate 3D vector combinations to calculate independent joint angles. To increase reliability, future iterations could integrate multiple geometric calculation sources simultaneously, cross-verifying values to eliminate erroneous readings.

Additionally, introducing an Inverse Kinematics (IK) framework is a realistic progression for this architecture. Instead of mirroring human joint angles directly, the vision pipeline would track only the operational coordinates (x, y, z) of the fingertips. While this approach adds slight mathematical complexity and a modest computational overhead, it allows the controller to calculate optimal servo configurations, compensating for mechanical tolerances and producing smoother trajectories.

4.1.3 Hardware Enhancements and Tactile Feedback

The ORCA hand currently operates as an open-loop system with no environmental feedback. To transition towards closed-loop manipulation, the following hardware upgrades are viable:

- **Tactile Feedback Integration:** Integrating tactile force sensors on the fingertips would enable active contact regulation. This enhancement is already featured on the latest official iteration of the ORCA hand hardware, though it introduces an additional component cost.

- **Smart Serial Bus Servos:** Upgrading standard PWM actuators to digital smart servos (e.g., Dynamixel protocol) is an achievable alternative. Despite the increased financial cost, it provides real-time hardware telemetry, allowing the micro-controller to monitor exact shaft positions, internal temperatures, and motor load.

4.2 Final Summary

The ORCA project successfully demonstrates a low-latency, biomimetic teleoperation system. The complete pipeline tracks the user’s hand gestures via computer vision, calculates real-time joint angles through geometric analysis, and translates these movements into physical commands to control the 16-DOF robotic hand. By utilizing edge processing, the system achieves responsive tracking without requiring high-end external computing hardware.

The integration process highlighted several practical challenges in hardware-software coordination. Adapting the code to the specific requirements of a spatial depth camera required adapting the initial acquisition pipeline. Furthermore, encountering physical power constraints and battery safety shutdowns under simultaneous motor loads forced the implementation of firmware optimization and power management routines to protect the mechanical structure.

Ultimately, the implementation of the hybrid dual-mode control framework validates the prototype. The bounded serial framing ensures data transmission integrity during AI control, while the physical MIDI interface serves as an essential debugging and calibration tool. This completed system fulfills the initial project objectives and establishes a reliable foundation for future advancements in closed-loop robotics.

5. Appendices

5.1 Kinematics and Geometric Core Functions

The following Python function (Listing 5.1) details the vector-based geometric framework used to compute finger joint flexions from the 3D spatial positions captured by MediaPipe.

```
1 def angle_flexion(PosA, PosB, PosC):
2     """
3     Calculates the flexion angle at a joint center (B) using two
4     bounding landmarks (A and C).
5     Computes the angle in degrees between vectors BA and BC.
6     """
7     # 1. Convert raw spatial coordinates into NumPy arrays for
8     # vector operations
9     A = np.array(PosA)
10    B = np.array(PosB)
11    C = np.array(PosC)
12
13    # 2. Define the two segments meeting at the joint pivot (B) as
14    # directional vectors
15    BA = A - B
16    BC = C - B
17
18    # 3. Compute the Euclidean norm (magnitude) for each vector
19    NormBA = np.linalg.norm(BA)
20    NormBC = np.linalg.norm(BC)
21
22    # 4. Compute the standard dot product between the two vectors
23    DOT_prod = np.dot(BA, BC)
24
25    # 5. Isolate the cosine of the angle between vectors
26    CosAngle = DOT_prod / (NormBA * NormBC)
27
28    # 6. Critical safety step: clip values to [-1.0, 1.0] to
29    # prevent floating-point precision errors from causing out-of-
30    # bounds inputs for the arccos function
31    CosAngle = np.clip(CosAngle, -1.0, 1.0)
32
33    # 7. Compute the inverse cosine and convert the result from
34    # radians to degrees
35    Angle = np.degrees(np.arccos(CosAngle))
36
37    # 8. Cast the final float value to an integer for optimized
38    # Serial transmission
39    return int(Angle)
```

Listing 5.1: Vector-based joint flexion angle calculation

5.2 Abduction and Sign Determination Algorithm

The Python implementation detailed in Listing 5.2 evaluates the lateral spreading of the fingers and includes the 2D cross-product determinant used to dynamically sign the middle finger's lateral deviation.

```
1 def angle_abduction(PosPalm, PosMiddleFinger, PosBase,
2   PosFirstJoint, MiddleFinger):
3     """
4     Calculates the abduction angle of a finger relative to the
5     central palm axis.
6     Applies a 2D determinant constraint to resolve directional
7     ambiguity for the middle finger.
8     """
9     # 1. Convert spatial landmark coordinates into NumPy arrays
10    Palm = np.array(PosPalm)
11    Mid = np.array(PosMiddleFinger)
12    Base = np.array(PosBase)
13    First = np.array(PosFirstJoint)
14
15    # 2. Define the structural reference vectors
16    Palm_Mid = Mid - Palm          # Central axis of the palm (Wrist
17    to Middle MCP)
18    Base_First = First - Base      # Directional vector of the
19    targeted finger
20
21    # 3. Compute vector magnitudes (Euclidean norms)
22    NormPM = np.linalg.norm(Palm_Mid)
23    NormBF = np.linalg.norm(Base_First)
24
25    # 4. Calculate the dot product and isolate the cosine value
26    DOT_prod = np.dot(Base_First, Palm_Mid)
27    CosAngle = DOT_prod / (NormPM * NormBF)
28
29    # 5. Prevent floating-point out-of-bound errors
30    CosAngle = np.clip(CosAngle, -1.0, 1.0)
31
32    # 6. Compute the absolute angle in degrees
33    Angle = np.degrees(np.arccos(CosAngle))
34
35    # 7. Directional sign assignment for the middle finger
36    if MiddleFinger:
37        # Compute the 2D determinant (X and Y components of the
38        cross product)
39        determinant = (Palm_Mid[0] * Base_First[1]) - (Palm_Mid[1]
40        * Base_First[0])
41
42        # A positive determinant indicates deviation to one side,
43        negative to the other
44        if determinant > 0:
45            return int(Angle)
46        else:
```

```

39         return -int(Angle)
40     else:
41         # Standard fingers only return unsigned magnitude
42         return int(Angle)

```

Listing 5.2: Vector-based joint abduction calculation with sign determination

5.3 Thumb Opposition Algorithm

The Python implementation detailed in Listing 5.3 evaluates the coordinate spacing between the thumb axis and the base of the metacarpal hierarchy to extract the opposition angle.

```

1  def angle_opposition(PosA, PosB, PosLittleFinger):
2      """
3      Calculates the thumb opposition angle relative to the base of
4      the little finger.
5      Uses the dot product of the palm span vector and the thumb
6      proximal segment.
7      """
8      # 1. Convert spatial landmarks into NumPy arrays
9      A = np.array(PosA)          # Thumb base (Landmark 1)
10     B = np.array(PosB)          # Thumb first joint (Landmark 2)
11     L = np.array(PosLittleFinger) # Little finger base (Landmark
12                                     17)
13
14     # 2. Define directional vectors according to the structural
15     triangle
16     AL = A - L                  # Vector tracking palm width
17     span
18     AB = B - A                  # Vector tracking thumb
19     alignment
20
21     # 3. Compute Euclidean norms
22     NormAL = np.linalg.norm(AL)
23     NormAB = np.linalg.norm(AB)
24
25     # 4. Calculate the dot product and isolate the cosine value
26     DOT_prod = np.dot(AL, AB)
27     CosAngle = DOT_prod / (NormAL * NormAB)
28
29     # 5. Prevent floating-point out-of-bounds precision errors
30     CosAngle = np.clip(CosAngle, -1.0, 1.0)
31
32     # 6. Compute the absolute angle in degrees and cast to integer
33     Angle = np.degrees(np.arccos(CosAngle))
34
35     return int(Angle)

```

Listing 5.3: Vector-based thumb opposition calculation

5.4 Multi-Joint Deadband Filter Algorithm

The Python function detailed in Listing 5.4 filters structural signal noise by establishing a static threshold comparison across all mapped joint angles before telemetry transmission.

```
1 def Dead_Band(List0, List1):
2     """
3     Applies a 3-degree deadband filter across an array of
4     calculated joint angles.
5     Prevents servo jittering by suppressing minor spatial
6     fluctuations.
7     List0: Array of validated angles from the previous frame.
8     List1: Array of raw angles computed from the current frame.
9     """
10    ListF = []
11
12    # Iterate through each joint angle in the dataset
13    for i in range(len(List0)):
14        # Check if the absolute angular variation exceeds the 3-
15        # degree threshold
16        if abs(List0[i] - List1[i]) > 3:
17            # Significant movement detected: accept and append the
18            # new angle
19            ListF.append(List1[i])
20        else:
21            # Minor fluctuation: maintain the previous angle to
22            # stabilize the motor
23            ListF.append(List0[i])
24
25    return ListF
```

Listing 5.4: Deadband threshold filter for servo jitter mitigation

5.5 Python Data Sorting and PWM Mapping Algorithms

The Python functions detailed in Listing 5.5 perform the structural reindexing and the linear interpolation mapping required to convert virtual joint angles into calibrated hardware commands.

```
1 def sort_ORCA(Sort_order, ToBeSorted_list):
2     """
3     Reorders the computed angle list to strictly align with the
4     physical
5     PCA9685 driver channel indexing layout (channels 0 to 15).
6     """
7     Sorted_list = []
8     for n in range(len(Sort_order)):
9         for index in Sort_order:
10             if n == index:
```



```

10         Sorted_list.append(ToBeSorted_list[Sort_order.index
11                               (index)])
12     return Sorted_list
13
14 def Angle_to_PWM(Sorted_Angle):
15     """
16     Translates sorted joint angles into hardware-specific 12-bit
17     PWM values.
18     Uses unique empirical ranges and handle directional inversions
19     per servo.
20     """
21     Sorted_PWM = []
22
23     # Mapping structure: np.interp(angle, [min_deg, max_deg], [
24     # min_pwm, max_pwm])
25     Sorted_PWM.append(int(np.interp(Sorted_Angle[0], [50, 179],
26                                     [235, 435])))
27     Sorted_PWM.append(int(np.interp(Sorted_Angle[1], [45, 179],
28                                     [215, 405])))
29     Sorted_PWM.append(int(np.interp(Sorted_Angle[2], [50, 179],
30                                     [235, 435])))
31     Sorted_PWM.append(int(np.interp(Sorted_Angle[3], [10, 40],
32                                     [225, 305])))
33     Sorted_PWM.append(int(np.interp(Sorted_Angle[4], [-20, 15],
34                                     [355, 265]))) # Inverted
35     Sorted_PWM.append(int(np.interp(Sorted_Angle[5], [5, 25],
36                                     [355, 235]))) # Inverted
37     Sorted_PWM.append(int(np.interp(Sorted_Angle[6], [45, 179],
38                                     [225, 405])))
39     Sorted_PWM.append(int(np.interp(Sorted_Angle[7], [50, 179],
40                                     [195, 385])))
41     Sorted_PWM.append(int(np.interp(Sorted_Angle[8], [135,
42                                     179], [195, 385])))
43     Sorted_PWM.append(int(np.interp(Sorted_Angle[9], [85, 179],
44                                     [195, 405])))
45     Sorted_PWM.append(int(np.interp(Sorted_Angle[10], [5, 40],
46                                     [255, 355])))
47     Sorted_PWM.append(int(np.interp(Sorted_Angle[11], [95, 140],
48                                     [355, 195]))) # Inverted
49     Sorted_PWM.append(int(np.interp(Sorted_Angle[12], [5, 40],
50                                     [345, 245]))) # Inverted
51     Sorted_PWM.append(int(np.interp(Sorted_Angle[13], [50, 179],
52                                     [205, 405])))
53     Sorted_PWM.append(int(np.interp(Sorted_Angle[14], [45, 179],
54                                     [235, 405])))
55     Sorted_PWM.append(int(np.interp(Sorted_Angle[15], [45, 179],
56                                     [235, 415])))
57
58     return Sorted_PWM

```

Listing 5.5: Reindexing and empirical PWM mapping implementations

5.6 Teensy Calibration and Hardware Interfacing Firmware

The following Arduino C++ firmware (Listing 5.6) is implemented on the Teensy microcontroller. Adapted from the supervisor’s core framework, it commands the PCA9685 driver via the Wire I2C library and provides interactive serial telemetry to isolate and calibrate each actuator.

```
1 #include <Arduino.h>
2 #include <Wire.h>
3 #include <Adafruit_PWMServoDriver.h>
4
5 // Initialize PCA9685 at standard I2C address 0x40
6 Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(0x40);
7 #define SERVO_FREQ 50
8
9 int activeServo = 0;
10 int currentPWM = 295; // Shared baseline midpoint position
11
12 void setup() {
13     Serial.begin(115200);
14     while (!Serial) { delay(10); } // Wait for serial monitor
15                                     connection
16
17     Wire.begin();
18     pwm.begin();
19     pwm.setPWMFreq(SERVO_FREQ);
20
21     // Safety startup protocol: cut power to all 16 channels
22     immediately
23     for (int i = 0; i < 16; i++) {
24         pwm.setPWM(i, 0, 4096);
25     }
26
27     // Initialize the first targeted actuator channel
28     pwm.setPWM(activeServo, 0, currentPWM);
29 }
30
31 void loop() {
32     if (Serial.available() > 0) {
33         String input = Serial.readStringUntil('\n');
34         input.trim(); // Strip invisible carriage return formatting
35
36         if (input.length() > 0) {
37             bool signalCoupe = false;
38
39             // Handle step-based movement adjustments
40             if (input == "a" || input == "A") currentPWM += 10;
41             else if (input == "z" || input == "Z") currentPWM -= 10;
42             else if (input == "q" || input == "Q") currentPWM += 1;
43             else if (input == "s" || input == "S") currentPWM -= 1;
```

```

43 // Hardware shutdown command: safelist the current channel to
    prevent heating
44 else if (input == "x" || input == "X") {
45     pwm.setPWM(activeServo, 0, 4096); // 4096 turns off the PWM
        pulse entirely
46     signalCoupe = true;
47 }
48
49 // Channel reassignment handling (numeric inputs)
50 else {
51     int targetServo = input.toInt();
52     if (targetServo >= 0 && targetServo <= 15 && (input == "0"
        || targetServo != 0)) {
53         // Put the previous channel to sleep before switching
54         pwm.setPWM(activeServo, 0, 4096);
55         activeServo = targetServo;
56         currentPWM = 295; // Reset to safe center pulse for the
            new channel
57     }
58 }
59
60 // Hardcoded software safety boundaries to protect mechanical
    limits
61 if (currentPWM < 50) currentPWM = 50;
62 if (currentPWM > 600) currentPWM = 600;
63
64 // Write parameters to the register if the signal is active
65 if (!signalCoupe) {
66     pwm.setPWM(activeServo, 0, currentPWM);
67 }
68 }
69 }
70 }

```

Listing 5.6: Interactive servo calibration and safety sleep firmware

5.7 Python Serial Transmitter Pipeline (Raspberry Pi)

The following Python function from the main tracking script handles the string conversion, packet formatting, and non-blocking streaming of the 16 joint parameters over the serial interface.

```

1 def Send_to_teenasy(PWM_list):
2     if teenasy is not None and teenasy.is_open:
3
4         # Convert integers to string and join with commas
5         INT_to_STR = ",".join(str(INT) for INT in PWM_list)
6         Complete_frame = f"<{INT_to_STR}>\n"
7
8         try:
9             teenasy.write(Complete_frame.encode('utf-8'))

```

```

10     except serial.SerialTimeoutException:
11         # If the Teensy buffer is full, we just drop the frame
           and move on
12         # This prevents OpenCV from freezing!
13         pass

```

Listing 5.7: Serial frame transmission and timeout safety inside the Python pipeline

5.8 Teensy Serial Receiver and Parser Firmware

The firmware implementation below processes incoming serial bytes character-by-character and extracts the integer values using string accumulation.

```

1 void readSerialCommand() {
2     static String buffer = "";
3     static int serialServoIndex = 0;
4
5     while (Serial.available() > 0) {
6         char actualRead = Serial.read();
7
8         if (actualRead == '<') {
9             // Start of a new frame: reset index and empty the buffer
10            serialServoIndex = 0;
11            buffer = "";
12        }
13        else if (actualRead >= '0' && actualRead <= '9') {
14            // It's a digit: append to the string buffer
15            buffer += actualRead;
16        }
17        else if (actualRead == ',') {
18            // Comma: end of one value. Convert string to int and store
               it.
19            if (serialServoIndex < NUM_SERVOS) {
20                servoCmd[serialServoIndex] = buffer.toInt();
21                serialServoIndex++;
22            }
23            buffer = "";
24        }
25        else if (actualRead == '>') {
26            // End of the frame: save the very last value
27            if (serialServoIndex < NUM_SERVOS) {
28                servoCmd[serialServoIndex] = buffer.toInt();
29            }
30            buffer = "";
31            serialServoIndex = 0;
32        }
33    }
34 }

```

Listing 5.8: Asynchronous serial packet parsing and buffer conversion logic

5.9 Teensy MIDI Manual Override Firmware

The code below handles the USB MIDI Host connection, control map channel routing, mode arbitration logic, and the automatic thermal protection cutoff timeout.

```
1 void setServoFromCC(uint8_t servoIndex, uint8_t ccValue) {
2   if (servoIndex >= NUM_SERVOS) return;
3   int target = map(ccValue, 0, 127, servoMin[servoIndex], servoMax[
4     servoIndex]);
5   servoCmd[servoIndex] = target;
6 }
7
8 void handleControlChange(byte channel, byte control, byte value) {
9   // MODE SWITCH BUTTON (CC 41 is the Play/Rec button)
10  if (control == 41 && value > 0) {
11    isManualMode = !isManualMode;
12    if (isManualMode) {
13      Serial.println("Switched to MANUAL MODE (KORG)");
14    } else {
15      Serial.println("Switched to AI VISION MODE (USB Serial)");
16    }
17    return;
18  }
19  // Reset Button (CC 42)
20  if (control == 42 && value > 0) {
21    initServoCommand();
22    return;
23  }
24
25  if (!isManualMode) return;
26
27  int servoIndex = -1;
28  for (int i = 0; i < 8; ++i) {
29    if (control == sliderCC[i]) { servoIndex = i; break; }
30  }
31  if (servoIndex < 0) {
32    for (int i = 0; i < 8; ++i) {
33      if (control == knobCC[i]) { servoIndex = 8 + i; break; }
34    }
35  }
36
37  if (servoIndex >= 0) {
38    setServoFromCC((uint8_t)servoIndex, value);
39  }
40 }
41
42 // Inside main loop routine
43 void loop() {
44   myusb.Task();
45   midi1.read();
46 }
```

```

47  if (!isManualMode) {
48      readSerialCommand();
49  } else {
50      while (Serial.available() > 0) {
51          Serial.read(); // Empty the hardware buffer so the Pi doesn't
                           freeze
52      }
53  }
54
55  for (int i = 0; i < NUM_SERVOS; ++i) {
56      if (servoCmd[i] != lastServoCmd[i]) {
57          pwm.setPWM(i, 0, servoCmd[i]);
58          lastServoCmd[i] = servoCmd[i];
59          lastMoveTime[i] = millis();
60          delay(2); // Cascade delay minimizing battery current spikes
61      }
62      else if ((millis() - lastMoveTime[i] > 500) && (lastServoCmd[i]
          != 4096)) {
63          pwm.setPWM(i, 0, 4096); // Thermal protection cutoff
                                   threshold
64          lastServoCmd[i] = 4096;
65      }
66  }
67  }

```

Listing 5.9: Teensy firmware handling MIDI control surface tracking and automatic power shutdown

6. Bibliography

- [Google AI Edge - MediaPipe Hand Landmarker documentation](#)